

Introduction To Neural Networks: A Simple Neural Network for Digit Recognition

Noah Robertson

Senior at the University of South Carolina, McNair Junior Fellows Awardee



UNIVERSITY OF
South Carolina

Introduction

Artificial Intelligence is on the rise in a never-before-seen way. With the release of mainstream large-language models, AI is one of the most talked about and researched subjects.

My goal over this past summer has been to seamlessly begin to grasp the foundations of AI and to find out what exactly is happening with/inside *Neural Networks*.

Neural Networks

Neural Networks are the building blocks of deep learning systems. These networks are composed of layers which contain neurons. Each neuron contains an activation value which constitutes to it “firing”.

The layers follow this format:

- An Input Layer – each neuron corresponds to some data value of the input
- Hidden Layer(s) – each neuron corresponds to an activation value
- An Output Layer – each neuron corresponds to an output percentage

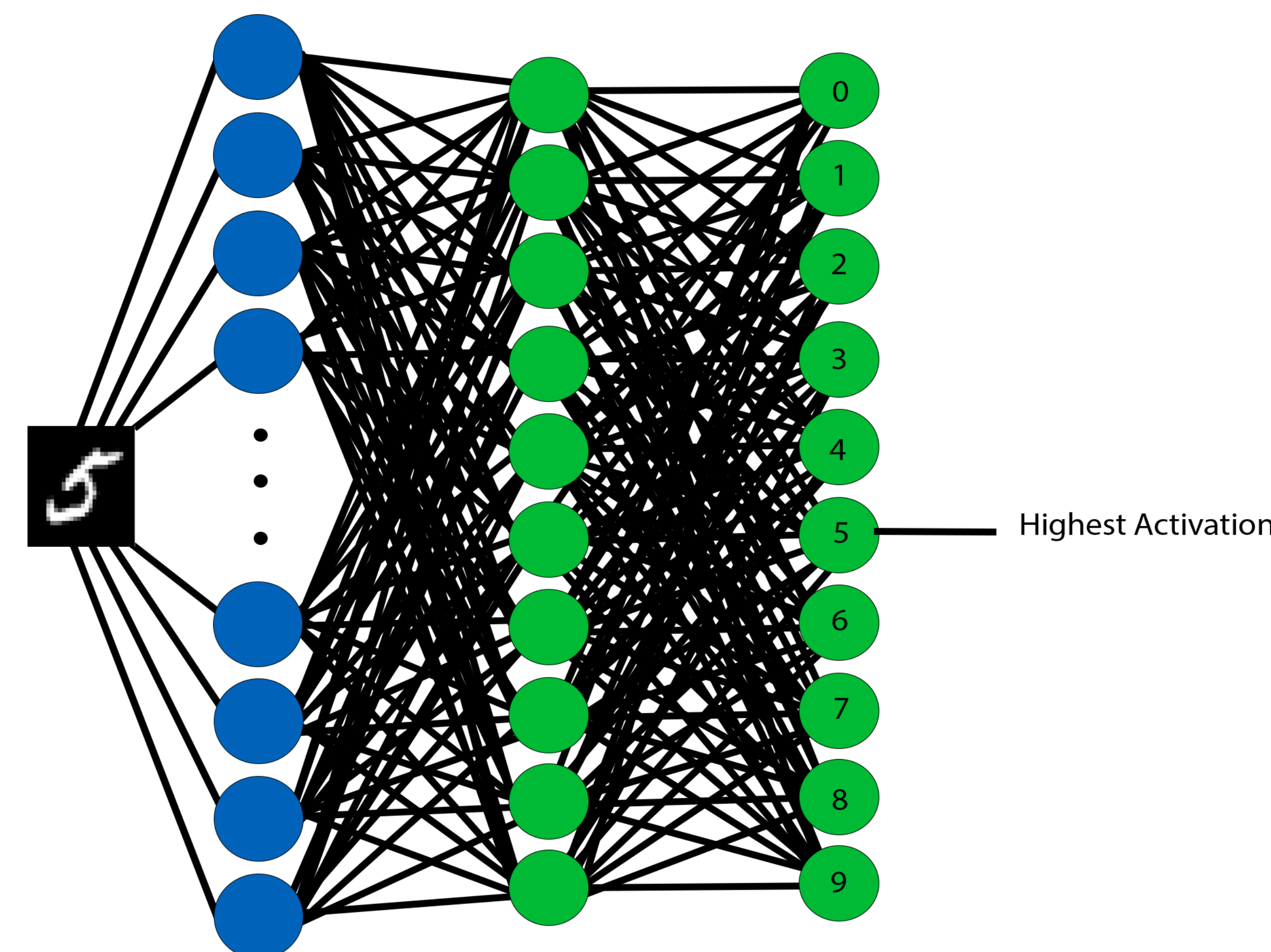
Each neuron is connected to all the neurons in the next layer by edges which are assigned certain weight values. Also, each neuron in every layer is assigned a bias value which corresponds to the threshold for the activation of that neuron.

Using the weights and biases as well as the input values, we calculate the activation value of each neuron using an activation function. This activation function keeps the activation value between -1 and 1, or sometimes 0 and 1. One of the most used activation functions is that of the *Sigmoid Function*:

$$\sigma(z) \equiv \frac{1}{1 + e^{-z}} \equiv \frac{1}{1 + e^{(-\sum_j w_j x_j - b)}}$$

Digit Recognition

One of the most tackled instances of neural networking is Digit Recognition. Thanks to Yann LeCun, there is open access to a large data set of handwritten digits and labels known as the MNIST digit data set. Using the MNIST data set we can implement a neural network using any programming language and train it with the images and labels. The basic architecture that I employed was the following:



The blue neurons represent the input neurons. The input is an image with grayscale pixel values 0 - 1, with zero being black and one being white. The image is 28x28 pixels in size, so the input layer consists of 784 total neurons.

The green neurons represent the hidden neurons and output neurons. These layers are of size ten each. The hidden neurons’ size is adjustable – the more neurons, the more accurate but slower. The output layer’s neurons correspond to the number outputted.

But how does this network correctly predict the input? With our matrices of weights and vectors of biases, we can employ strategies to modify these values based on a learning algorithm.

Learning

The bulk of neural networks is the process of learning. In order to get the network to accurately identify digits, we must employ a cost function. This cost function will quantify how accurate the neural network is. We want this cost function to approximate to zero.

In order to do so we must find sets of weights and biases that make the cost function as small as possible. To do so you can employ **stochastic gradient descent**. Gradient descent is the process of finding the lowest point on a theoretical 3D graph. Stochastic here means that we are taking smaller batches of training data and adjusting the weights and biases according to these mini-batches, instead of all the data.

Gradient descent employs the backpropagation algorithm. The backpropagation algorithm does the brunt of the work by calculating the gradients for changing the weights and biases. This process works using the label of the image to work backwards through the network to calculate these gradients.

So, the learning algorithm consists of:

1. Inputting a set of training examples.
2. For each training example:
 - a. Feedforward
 - b. Output Error
 - c. Backpropagate the Error
3. Gradient Descent



Conclusions

Neural Networks are complex, but once the basic concepts are grasped, they can be employed to start tackling problems. I created a simple neural network using Go that achieves an 80% accuracy rating and am still working on improving it.

I would also like to mention *Michael Nielsen's book on Neural Networks*. It has had a great impact on my research in this subject. Scan the QR code for a link to his book as well as the GitHub repository for my code. You can also access his code repository on his book’s website, which using the same layers and neurons, yields about a 94-95% accuracy.